

1. Use KNN to classify handwritten digits for MNIST dataset

Code:

```
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

mnist = fetch_openml('mnist_784', version=1, as_frame=False)
X, y = mnist.data, mnist.target.astype('int')

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=10000, random_state=42)

knn = KNeighborsClassifier(n_neighbors=3, weights='distance')
knn.fit(X_train, y_train)

y_pred = knn.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}")
```

OUTPUT: Accuracy: 0.9730

2. Implement Gradient Boosting on breast cancer dataset to predict whether a tumor is malignant or benign and evaluate performance metrics

Code:

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import accuracy_score, recall_score, precision_score, f1_score,
confusion_matrix, classification_report

# Load dataset
data = load_breast_cancer()
X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = GradientBoostingClassifier(n_estimators=100, max_depth=3, learning_rate=0.1,
random_state=42)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall :", recall)
print("F1 Score :", f1)

print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

OUTPUT

```
Accuracy : 0.956140350877193
Precision: 0.9583333333333334
Recall : 0.971830985915493
F1 Score : 0.965034965034965

Confusion Matrix:
[[40  3]
 [ 2 69]]

Classification Report:

```

	precision	recall	f1-score	support
0	0.95	0.93	0.94	43
1	0.96	0.97	0.97	71
accuracy			0.96	114
macro avg	0.96	0.95	0.95	114
weighted avg	0.96	0.96	0.96	114

3. Use t-SNE to visualise high dimensional data (e.g.; MNIST digits) in 2D

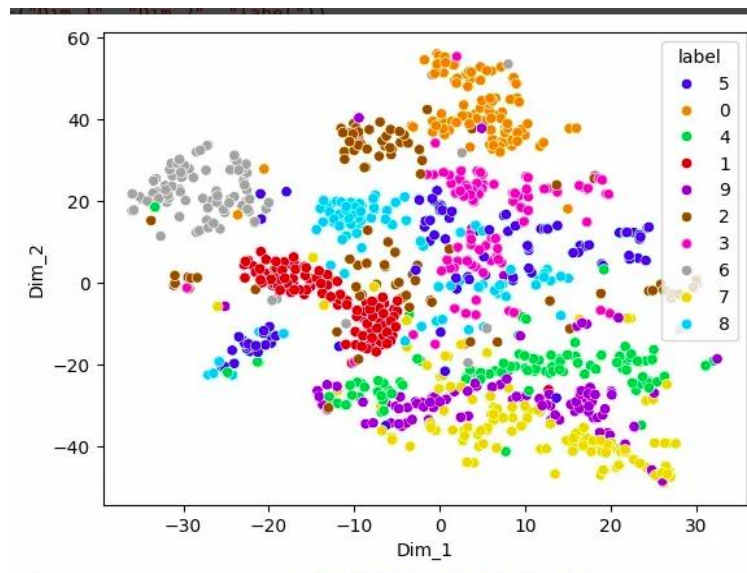
Code:

```
import numpy as np
import pandas as pd
import seaborn as sn
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import fetch_openml

mnist = fetch_openml('mnist_784', version=1)
d = mnist.data
l = mnist.target
df = pd.DataFrame(d)
df['label'] = l

X = df.drop('label', axis=1)
standardized_data = StandardScaler().fit_transform(X)

data_1000 = standardized_data[0:1000, :]
labels_1000 = l[0:1000]
model = TSNE(n_components = 2, random_state = 0)
tsne_data = model.fit_transform(data_1000)
tsne_data = np.vstack((tsne_data.T, labels_1000)).T
tsne_df = pd.DataFrame(data = tsne_data, columns = ("Dim_1", "Dim_2", "label"))
sn.scatterplot(data=tsne_df, x='Dim_1', y='Dim_2', hue='label', palette="bright")
plt.show()
```



4. Use global temperature data (e.g., temperature anomaly data for different countries) to group countries based on their temperature patterns. Apply Agglomerative Hierarchical Clustering and analyze how countries are clustered based on their temperature trends.

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.impute import SimpleImputer

print("Loading dataset...")

try:
    df = pd.read_csv('temperature_anomaly_detection.csv')
    print("Dataset loaded successfully.")
except FileNotFoundError:
    print("Error: 'temperature_anomaly_detection.csv' not found. Please ensure the file is in the current directory.")
df = pd.DataFrame(columns=['t1', 't2', 't3', 't4', 't5', 'label'])
except Exception as e:
    print(f"Error loading dataset: {e}")
    df = pd.DataFrame(columns=['t1', 't2', 't3', 't4', 't5', 'label'])

print("\nOriginal Columns:", df.columns)
print("\nFirst 5 rows of original data:\n", df.head())

feature_columns = ['t1', 't2', 't3', 't4', 't5']

if not all(col in df.columns for col in feature_columns):
    print(f"Error: Expected feature columns {feature_columns} not found in the DataFrame.")
    print("Please ensure 'temperature_anomaly_detection.csv' contains these columns or adjust the script.")
    df_pivot_imputed = pd.DataFrame()
    X_scaled = np.array([])
    countries = []
else:
    df_for_clustering = df[feature_columns].copy()
    df_for_clustering.dropna(inplace=True)

    countries = df_for_clustering.index.astype(str).tolist()

    df_pivot_imputed = df_for_clustering

print("\nShape of data for clustering (Entities x Features):", df_pivot_imputed.shape if not df_pivot_imputed.empty else "(0, 0)")
print("\nFirst 5 rows of data for clustering:\n", df_pivot_imputed.head())
```

```

if not df_pivot_imputed.empty and df_pivot_imputed.shape[0] > 1:
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(df_pivot_imputed)

    print("\nApplying Agglomerative Hierarchical Clustering...")

    linked = linkage(X_scaled, method='ward')
    plt.figure(figsize=(20, 10))
    dendrogram(linked,
                orientation='top',
                labels=list(countries),
                distance_sort='descending',
                show_leaf_counts=True)
    plt.title('Hierarchical Clustering Dendrogram (Patterns based on t1-t5 features)')
    plt.xlabel('Entities (Row Index)')
    plt.ylabel('Euclidean Distance')
    plt.tight_layout()
    plt.show()

    n_clusters = 6

    if n_clusters > X_scaled.shape[0]:
        n_clusters = X_scaled.shape[0]
        print(f'Adjusted number of clusters to {n_clusters} as it was greater than the number
              of samples.")
    if n_clusters < 1:
        print("Cannot perform clustering with less than 1 cluster or no data.")
    else:
        agg_clustering = AgglomerativeClustering(n_clusters=n_clusters,
metric='euclidean', linkage='ward')
        clusters = agg_clustering.fit_predict(X_scaled)

        entity_clusters = pd.DataFrame({'Entity_ID': countries, 'Cluster': clusters})
        print(f"\nEntities grouped into {n_clusters} clusters:")
        for i in range(n_clusters):
            print(f"\nCluster {i}:")
            print(entity_clusters[entity_clusters['Cluster'] == i]['Entity_ID'].tolist())
            df_pivot_imputed['Cluster'] = clusters
        cluster_means = df_pivot_imputed.groupby('Cluster').mean()

        plt.figure(figsize=(15, 8))
        for i in range(n_clusters):
            plt.plot(cluster_means.loc[i], label=f'Cluster {i} Mean', alpha=0.7)
            plt.title('Mean Feature Pattern for Each Cluster (t1-t5)')
            plt.xlabel('Feature (t1-t5)')
            plt.ylabel('Mean Feature Value')
            plt.legend()
            plt.grid(True)
            plt.tight_layout()
            plt.show()

```

```

print("\nCluster means (feature patterns over t1-t5):")
print(cluster_means.head())

df_pivot_imputed.drop(columns=['Cluster'], inplace=True)
else:
    print("Skipping clustering and analysis due to insufficient data for processing.")

```

OUTPUT

Loading dataset...

Dataset loaded successfully.

Original Columns: Index(['t1', 't2', 't3', 't4', 't5', 'label'], dtype='object')

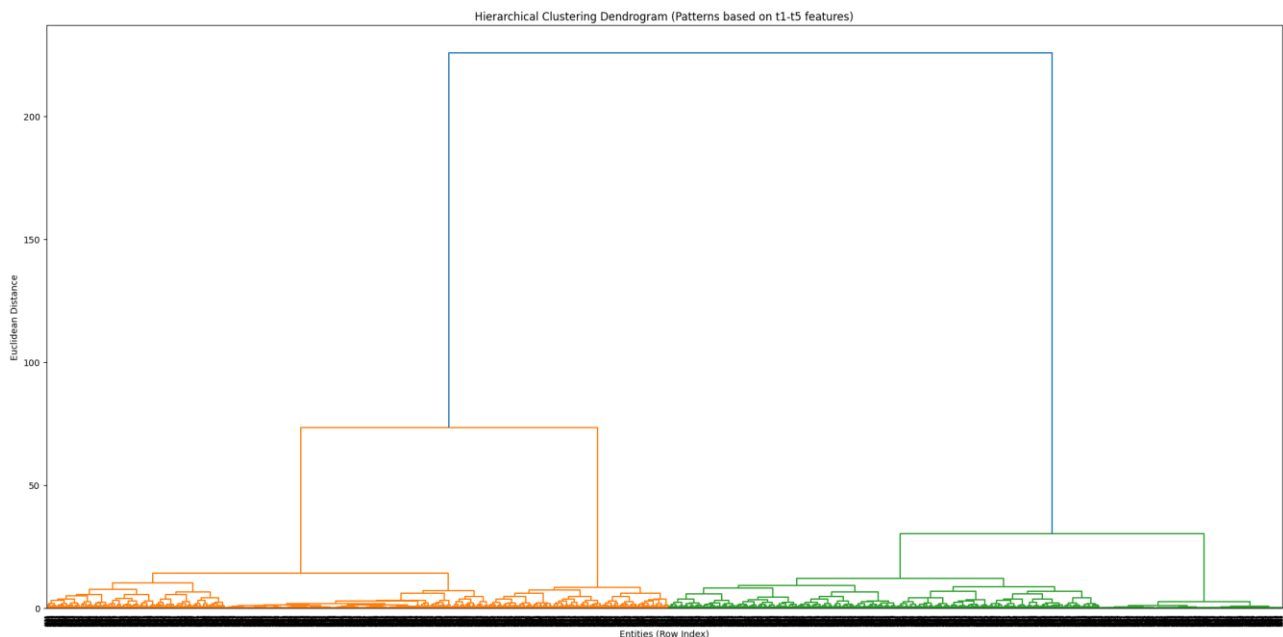
First 5 rows of original data:

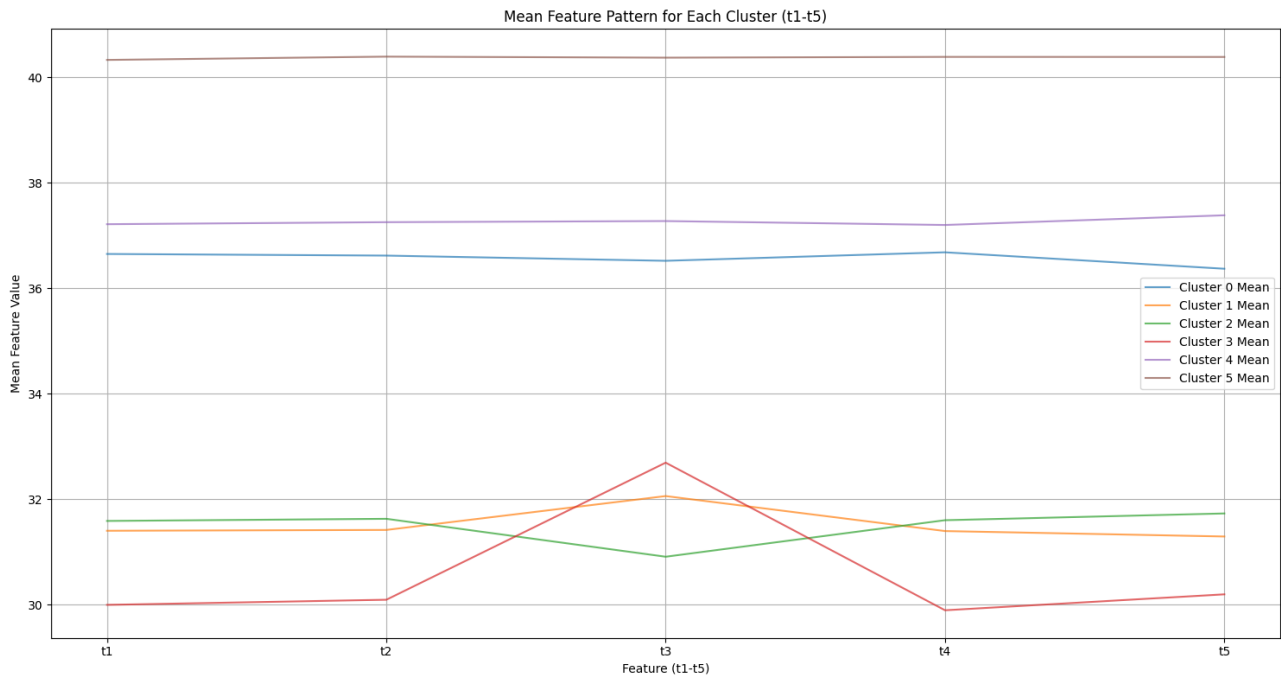
	t1	t2	t3	t4	t5	label
0	33.00	30.59	31.83	31.26	32.66	0
1	35.95	37.61	36.47	36.70	37.81	0
2	30.10	30.20	32.73	29.98	30.28	1
3	36.36	37.11	35.73	36.46	38.25	1
4	39.61	39.80	39.19	41.57	40.38	1

Shape of data for clustering (Entities x Features): (6000, 5)

First 5 rows of data for clustering:

	t1	t2	t3	t4	t5
0	33.00	30.59	31.83	31.26	32.66
1	35.95	37.61	36.47	36.70	37.81
2	30.10	30.20	32.73	29.98	30.28
3	36.36	37.11	35.73	36.46	38.25
4	39.61	39.80	39.19	41.57	40.38





Cluster means (feature patterns over t1-t5):

	t1	t2	t3	t4	t5
Cluster					
0	36.653958	36.622882	36.524167	36.685532	36.373565
1	31.405304	31.421097	32.063427	31.399855	31.296618
2	31.591820	31.632315	30.913084	31.605733	31.733660
3	30.000620	30.097531	32.696584	29.898117	30.199944
4	37.218296	37.256693	37.277866	37.203636	37.387287

5. Use the Heart Disease dataset to predict whether a patient has heart disease or not. Implement Stratified K-Fold Cross-Validation with a classifier like SVM and evaluate the model performance.

Code:

```
import pandas as pd
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.svm import SVC
from sklearn.model_selection import StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
roc_auc_score, confusion_matrix

data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target)
print("Dataset loaded successfully.")
print(f"Features shape: {X.shape}, Target shape: {y.shape}")
print("\nFirst 5 rows of features:\n", X.head())
print("\nFirst 5 values of target:\n", y.head())

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

svm = SVC(kernel='linear', probability=True, random_state=42) #

n_splits = 5
skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=42)

accuracy_scores = []
precision_scores = []
recall_scores = []
f1_scores = []
roc_auc_scores = []
conf_matrices = []

for fold, (train_index, test_index) in enumerate(skf.split(X_scaled, y)):
    X_train, X_test = X_scaled[train_index], X_scaled[test_index]
    y_train, y_test = y.iloc[train_index], y.iloc[test_index]

    svm.fit(X_train, y_train)
    y_pred = svm.predict(X_test)
    y_pred_proba = svm.predict_proba(X_test)[:, 1]

    accuracy_scores.append(accuracy_score(y_test, y_pred))
    precision_scores.append(precision_score(y_test, y_pred))
    recall_scores.append(recall_score(y_test, y_pred))
```



```

fl_scores.append(fl_score(y_test, y_pred))
roc_auc_scores.append(roc_auc_score(y_test, y_pred_proba))
conf_matrices.append(confusion_matrix(y_test, y_pred))

print(f"Fold {fold+1}: Accuracy = {accuracy_scores[-1]:.4f}, ROC AUC = {roc_auc_scores[-1]:.4f}")

print(f"Average Accuracy: {np.mean(accuracy_scores):.4f} (+/- {np.std(accuracy_scores):.4f})")
print(f"Average Precision: {np.mean(precision_scores):.4f} (+/- {np.std(precision_scores):.4f})")
print(f"Average Recall: {np.mean(recall_scores):.4f} (+/- {np.std(recall_scores):.4f})")
print(f"Average F1-Score: {np.mean(fl_scores):.4f} (+/- {np.std(fl_scores):.4f})")
print(f"Average ROC AUC: {np.mean(roc_auc_scores):.4f} (+/- {np.std(roc_auc_scores):.4f})")

print("\nAverage Confusion Matrix across folds:")
print(np.mean(conf_matrices, axis=0).astype(int))

```

OUTPUT

Dataset loaded successfully.

Features shape: (569, 30), Target shape: (569,)

First 5 rows of features:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness \
0	17.99	10.38	122.80	1001.0	0.11840
1	20.57	17.77	132.90	1326.0	0.08474
2	19.69	21.25	130.00	1203.0	0.10960
3	11.42	20.38	77.58	386.1	0.14250
4	20.29	14.34	135.10	1297.0	0.10030

	mean compactness	mean concavity	mean concave points	mean symmetry \
0	0.27760	0.3001	0.14710	0.2419
1	0.07864	0.0869	0.07017	0.1812
2	0.15990	0.1974	0.12790	0.2069
3	0.28390	0.2414	0.10520	0.2597
4	0.13280	0.1980	0.10430	0.1809

	mean fractal dimension	...	worst radius	worst texture	worst perimeter \
0	0.07871	...	25.38	17.33	184.60
1	0.05667	...	24.99	23.41	158.80
2	0.05999	...	23.57	25.53	152.50
3	0.09744	...	14.91	26.50	98.87
4	0.05883	...	22.54	16.67	152.20

	worst area	worst smoothness	worst compactness	worst concavity \
0	2019.0	0.1622	0.6656	0.7119
1	1956.0	0.1238	0.1866	0.2416
2	1709.0	0.1444	0.4245	0.4504
3	567.7	0.2098	0.8663	0.6869
4	1575.0	0.1374	0.2050	0.4000

worst concave points worst symmetry worst fractal dimension

0	0.2654	0.4601	0.11890
1	0.1860	0.2750	0.08902
2	0.2430	0.3613	0.08758
3	0.2575	0.6638	0.17300
4	0.1625	0.2364	0.07678

[5 rows x 30 columns]

First 5 values of target:

0 0

1 0

2 0

3 0

4 0

dtype: int64

Fold 1: Accuracy = 0.9912, ROC AUC = 1.0000

Fold 2: Accuracy = 0.9474, ROC AUC = 0.9954

Fold 3: Accuracy = 0.9561, ROC AUC = 0.9775

Fold 4: Accuracy = 0.9912, ROC AUC = 1.0000

Fold 5: Accuracy = 0.9912, ROC AUC = 0.9987

Average Accuracy: 0.9754 (+/- 0.0195)

Average Precision: 0.9756 (+/- 0.0247)

Average Recall: 0.9860 (+/- 0.0089)

Average F1-Score: 0.9806 (+/- 0.0153)

Average ROC AUC: 0.9943 (+/- 0.0086)

Average Confusion Matrix across folds:

[[40 1]

[1 70]]

6. Implement SARSA in a small environment to see how an agent learns step-by-step.

Code:

```
import numpy as np
import random
import time
from IPython.display import clear_output

# 1. Define the Environment (GridWorld)
class GridWorld:
    def __init__(self, size=4):
        self.size = size
        self.grid = np.zeros((size, size))
        self.start_state = (0, 0)
        self.goal_state = (size - 1, size - 1)
        self.trap_states = [] # Define any trap states here if needed
        self.state = self.start_state

    def reset(self):
        self.state = self.start_state
        return self.state

    def step(self, action):
        # Actions: 0: Up, 1: Down, 2: Left, 3: Right
        row, col = self.state
        if action == 0: # Up
            row = max(0, row - 1)
        elif action == 1: # Down
            row = min(self.size - 1, row + 1)
        elif action == 2: # Left
            col = max(0, col - 1)
        elif action == 3: # Right
            col = min(self.size - 1, col + 1)

        new_state = (row, col)

        reward = -0.1 # Small negative reward for each step to encourage shorter paths
        done = False

        if new_state == self.goal_state:
            reward = 10
            done = True
        elif new_state in self.trap_states:
            reward = -5
            done = True

        self.state = new_state
        return new_state, reward, done
```

```

def render(self):
    clear_output(wait=True)
    display_grid = np.full((self.size, self.size), '.')
    display_grid[self.goal_state] = 'G'
    for trap in self.trap_states:
        display_grid[trap] = 'T'
    display_grid[self.state] = 'A'

    print("Grid World:")
    for row in display_grid:
        print(" ".join(row))
    print(f'Current State: {self.state}')
    time.sleep(0.1)

```

2. SARSA Algorithm Implementation

```

def sarsa(env, num_episodes=1000, alpha=0.1, gamma=0.99, epsilon=0.1,
render_each_step=False):
    # Initialize Q-table with zeros
    Q = np.zeros((env.size, env.size, 4)) # State (row, col), Action (0-3)

    rewards_per_episode = []

    for episode in range(num_episodes):
        state = env.reset()
        state_idx = (state[0], state[1])
        done = False
        total_reward = 0

        # Choose initial action A using epsilon-greedy policy
        if random.uniform(0, 1) < epsilon:
            action = random.randint(0, 3) # Explore
        else:
            action = np.argmax(Q[state_idx]) # Exploit

        if (episode + 1) % (num_episodes // 10) == 0: # Print progress periodically
            print(f'Episode {episode + 1}/{num_episodes}')

        while not done:
            if render_each_step:
                env.render()

            next_state, reward, done = env.step(action)
            next_state_idx = (next_state[0], next_state[1])

            total_reward += reward

            # Choose next action A' using epsilon-greedy policy
            if random.uniform(0, 1) < epsilon:
                next_action = random.randint(0, 3) # Explore
            else:
                next_action = np.argmax(Q[next_state_idx]) # Exploit

```

```

# SARSA Update Rule
#  $Q(s,a) = Q(s,a) + \alpha * (\text{reward} + \gamma * Q(s',a) - Q(s,a))$ 
old_q_value = Q[state_idx + (action,)]
next_q_value = Q[next_state_idx + (next_action,)]

Q[state_idx + (action,)] = old_q_value + alpha * (reward + gamma * next_q_value -
old_q_value)

state = next_state
state_idx = next_state_idx
action = next_action

rewards_per_episode.append(total_reward)

return Q, rewards_per_episode

# 3. Run the SARSA Agent

# Create the environment
env = GridWorld(size=5)
# Add a trap state for demonstration
env.trap_states = [(2,2), (1,3)]

print("Training SARSA agent...")
Q_table, episode_rewards = sarsa(env, num_episodes=5000, alpha=0.1, gamma=0.9, epsilon=0.2,
render_each_step=False)
print("Training complete.")

# 4. Visualize Results (Optional: Show agent's path after learning)
print("\n--- Demonstrating learned path ---")
state = env.reset()
env.render()
path = [state]
time.sleep(0.5)
done = False
while not done:
    state_idx = (state[0], state[1])
    action = np.argmax(Q_table[state_idx]) # Always exploit learned Q-values
    state, reward, done = env.step(action)
    path.append(state)
    env.render()
    time.sleep(0.5)

print("\nLearned path:")
print(path)
print(f"Final Q-table (showing values for state {env.start_state}): \n{Q_table[env.start_state]}")

# Plot rewards over time
import matplotlib.pyplot as plt
plt.figure(figsize=(10, 6))
plt.plot(episode_rewards)

```

```
plt.title('SARSA: Rewards per Episode')
plt.xlabel('Episode')
plt.ylabel('Total Reward')
plt.grid(True)
plt.show()
```

OUTPUT:

Grid World:

```
.....
... T .
.. T ..
.....
.... A
```

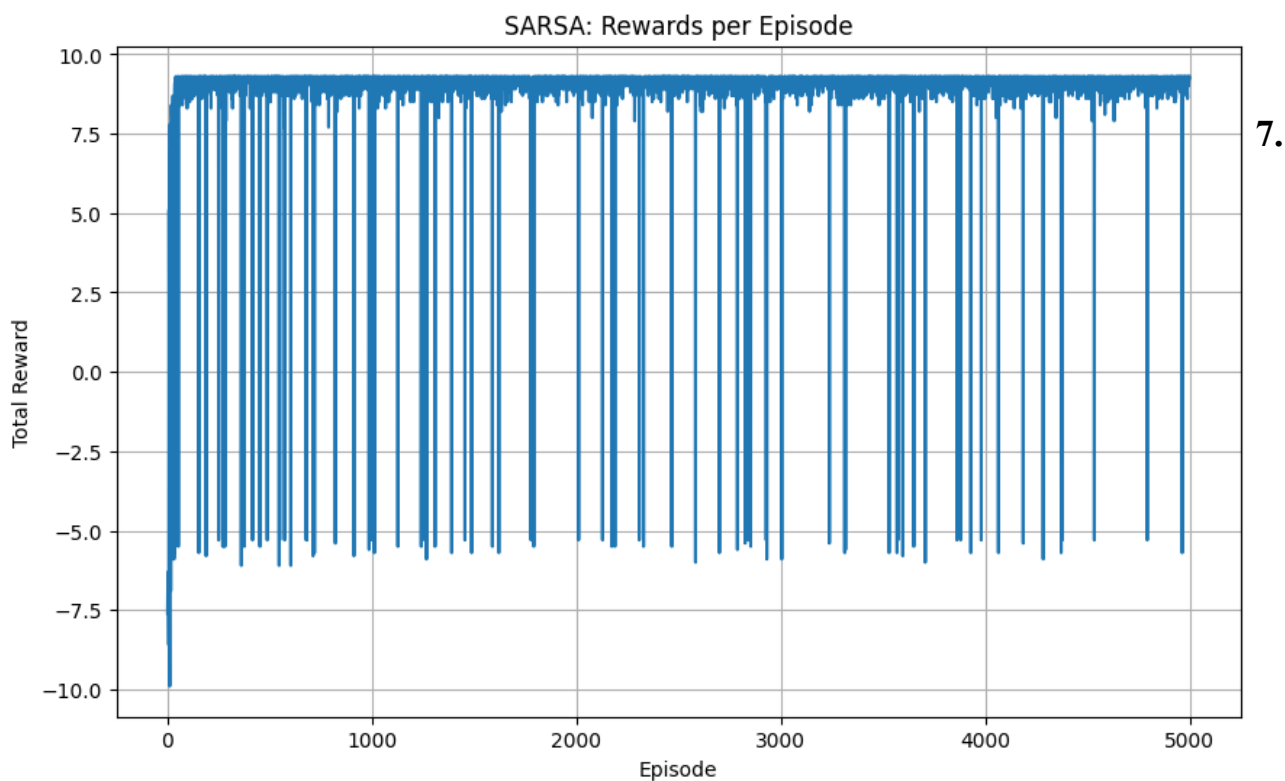
Current State: (4, 4)

Learned path:

```
[(0, 0), (1, 0), (2, 0), (3, 0), (4, 0), (4, 1), (4, 2), (4, 3), (4, 4)]
```

Final Q-table (showing values for state (0, 0)):

```
[2.77415151 3.43204536 2.8265396 2.2572774 ]
```



7. Taking a movie reviews dataset, use a pre-trained BERT model to classify text either positive or negative reviews.

Code

```
import pandas as pd
from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForSequenceClassification
import torch
from torch.nn.functional import softmax

# 1. Load the dataset
print("Loading IMDb movie reviews dataset...")
# Using a small subset of the test set for demonstration to keep execution quick
dataset = load_dataset("imdb")
sample_size = 10 # Number of reviews to sample for demonstration
test_reviews = dataset['test'].shuffle(seed=42).select(range(sample_size))

# 2. Load pre-trained tokenizer and model
print("\nLoading pre-trained DistilBERT tokenizer and sentiment classification model...")
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased-finetuned-sst-2-english")
model = AutoModelForSequenceClassification.from_pretrained("distilbert-base-uncased-finetuned-sst-2-english")

# Ensure model is in evaluation mode
model.eval()

# Move model to GPU if available
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
print(f"Using device: {device}")

# 3. Define a function to classify reviews
def classify_review(text):
    inputs = tokenizer(text, return_tensors="pt", truncation=True, padding=True, max_length=512)
    inputs = {key: val.to(device) for key, val in inputs.items()}

    with torch.no_grad():
        outputs = model(**inputs)

    predictions = softmax(outputs.logits, dim=1)
    # The model 'distilbert-base-uncased-finetuned-sst-2-english' typically outputs:
    # 0 for negative sentiment, 1 for positive sentiment.
    predicted_class_id = predictions.argmax().item()
    confidence = predictions.max().item()

    # Map class_id to labels
    label_map = {0: "Negative", 1: "Positive"}
    predicted_label = label_map[predicted_class_id]

    return predicted_label, confidence
```

```

# 4. Classify sample reviews from the dataset
print("\nClassifying sample movie reviews from the IMDb dataset:")
results = []
for i, example in enumerate(test_reviews):
    review_text = example['text']
    true_label = "Positive" if example['label'] == 1 else "Negative"
    predicted_label, confidence = classify_review(review_text)
    results.append({
        "Review_Index": i,
        "Review_Text_Snippet": review_text[:100] + "...",
        "True_Label": true_label,
        "Predicted_Label": predicted_label,
        "Confidence": f"{confidence:.4f}"
    })

results_df = pd.DataFrame(results)
print(results_df.to_string())

# 5. Demonstrate with custom reviews
print("\n--- Custom Review Classification ---")
custom_review_positive = "This movie was absolutely brilliant! The acting was superb and the plot was engaging from start to finish. Highly recommend!"
custom_review_negative = "What a waste of time. The plot was nonsensical, the acting was wooden, and I regret every minute I spent watching this."
custom_review_neutral = "The movie had some interesting parts but overall it was just okay, nothing special."

print(f"Review: \"{custom_review_positive}\"")
predicted_label, confidence = classify_review(custom_review_positive)
print(f"Predicted: {predicted_label} (Confidence: {confidence:.4f})")

print(f"Review: \"{custom_review_negative}\"")
predicted_label, confidence = classify_review(custom_review_negative)
print(f"Predicted: {predicted_label} (Confidence: {confidence:.4f})")

print(f"Review: \"{custom_review_neutral}\"")
predicted_label, confidence = classify_review(custom_review_neutral)
print(f"Predicted: {predicted_label} (Confidence: {confidence:.4f})")

```

OUTPUT:


```

plain_text/train-00000-of-00001.parquet: 100% ██████████ 21.0M/21.0M [00:01<00:00, 105MB/s]
plain_text/test-00000-of-00001.parquet: 100% ██████████ 20.5M/20.5M [00:00<00:00, 103MB/s]
plain_text/unsupervised-00000-of-00001.p(...): 100% ██████████ 42.0M/42.0M [00:00<00:00, 212MB/s]

Generating train split: 100% ██████████ 25000/25000 [00:00<00:00, 61769.76 examples/s]
Generating test split: 100% ██████████ 25000/25000 [00:00<00:00, 51110.13 examples/s]
Generating unsupervised split: 100% ██████████ 50000/50000 [00:00<00:00, 73309.15 examples/s]

Loading pre-trained DistilBERT tokenizer and sentiment classification model...
config.json: 100% ██████████ 629/629 [00:00<00:00, 30.3kB/s]
tokenizer_config.json: 100% ██████████ 48.0/48.0 [00:00<00:00, 1.29kB/s]
vocab.txt: █████ 232k/? [00:00<00:00, 5.34MB/s]
model.safetensors: 100% ██████████ 268M/268M [00:02<00:00, 1.35GB/s]
Loading weights: 100% ██████████ 104/104 [00:00<00:00, 302.44it/s, Materializing param=pre_classifier.weight]
Using device: cpu

```

Classifying sample movie reviews from the IMDb dataset:

Review_Index	Review_Text_Snippet	True_Label	Predicted_Label	Confidence
0	 When I unsuspectedly rented A Thousand Acres, I thought I was in for an entertaining Kin...	Positive	Positive	0.9989
1	This is the latest entry in the long series of films with the French agent, O.S.S. 117 (the French a...	Positive	Positive	0.9970
2	This movie was so frustrating. Everything seemed energetic and I was totally prepared to have a good...	Negative	Negative	0.9972
3	I was truly and wonderfully surprised at "O' Brother, Where Art Thou?" The video store was out of al...	Positive	Negative	0.6492
4	This movie spends most of its time preaching that it is the script that makes the movie, but apparen...	Negative	Negative	0.9985
5	After a very long time Marathi cinema has come with some good movie.This movie is one of the best Ma...	Positive	Positive	0.9997

--- Custom Review Classification ---

Review: "This movie was absolutely brilliant! The acting was superb and the plot was engaging from start to finish. Highly recommend!"

Predicted: Positive (Confidence: 0.9999)

Review: "What a waste of time. The plot was nonsensical, the acting was wooden, and I regret every minute I spent watching this."

Predicted: Negative (Confidence: 0.9998)

Review: "The movie had some interesting parts but overall it was just okay, nothing special."

Predicted: Negative (Confidence: 0.9298)

8. Use the California Housing dataset to predict house prices. Build a regression model using TensorFlow to predict the price of a house based on features. Train the model and evaluate its performance using Mean Squared Error (MSE) metrics.

Code:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from sklearn.metrics import mean_squared_error
import matplotlib.pyplot as plt

print("Loading California Housing dataset from local CSVs...")
# Load the California Housing dataset from local CSV files
train_df = pd.read_csv('/content/sample_data/california_housing_train.csv')
test_df = pd.read_csv('/content/sample_data/california_housing_test.csv')

# Combine train and test data for consistent preprocessing and splitting
df_combined = pd.concat([train_df, test_df], ignore_index=True)

# Separate features (X) and target (y)
X = df_combined.drop(columns=['median_house_value'])
y = df_combined['median_house_value']

print("Dataset loaded successfully.")
print(f"Features shape: {X.shape}, Target shape: {y.shape}")
print("\nFirst 5 rows of features:\n", X.head())
print("\nFirst 5 values of target:\n", y.head())

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print(f"\nTraining set shape: {X_train.shape}, Test set shape: {X_test.shape}")

# Standardize features
print("\nStandardizing features...")
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Build the TensorFlow regression model
print("\nBuilding TensorFlow regression model...")
model = Sequential([
    Dense(64, activation='relu', input_shape=(X_train_scaled.shape[1],)),
    Dense(32, activation='relu'),
    Dense(1) # Output layer for regression (single continuous value)
])
```

```

# Compile the model
model.compile(optimizer='adam', loss='mean_squared_error')
model.summary()

# Train the model
print("\nTraining the model...")
history = model.fit(
    X_train_scaled, y_train,
    epochs=50, # Number of training iterations
    batch_size=32,
    validation_split=0.1, # Use a portion of training data for validation
    verbose=1
)

# Evaluate the model
print("\nEvaluating the model on the test set...")
y_pred = model.predict(X_test_scaled)
mse = mean_squared_error(y_test, y_pred)
print(f"Mean Squared Error (MSE) on the test set: {mse:.4f}")

# Plot training history
plt.figure(figsize=(10, 6))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss During Training')
plt.xlabel('Epoch')
plt.ylabel('Mean Squared Error')
plt.legend()
plt.grid(True)
plt.show()

# Visualize predictions vs. actual values (sample)
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred, alpha=0.3)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')
plt.xlabel('Actual Prices')
plt.ylabel('Predicted Prices')
plt.title('Actual vs. Predicted House Prices')
plt.grid(True)
plt.show()

```

OUTPUT

```

Training the model...
Epoch 1/50
450/450 ————— 6s 7ms/step - loss: 55951917056.0000
- val_loss: 56151547904.0000
Epoch 2/50
450/450 ————— 4s 9ms/step - loss: 53715021824.0000
- val_loss: 51745460224.0000

```

Epoch 3/50
450/450 ————— 3s 6ms/step - loss: 46952566784.0000
- val_loss: 42617905152.0000
Epoch 4/50
450/450 ————— 5s 7ms/step - loss: 36434337792.0000
- val_loss: 31382056960.0000
Epoch 5/50
450/450 ————— 5s 6ms/step - loss: 25637324800.0000
- val_loss: 21896988672.0000
Epoch 6/50
450/450 ————— 4s 4ms/step - loss: 17769805824.0000
- val_loss: 16206673920.0000
Epoch 7/50
450/450 ————— 2s 4ms/step - loss: 13668091904.0000
- val_loss: 13660295168.0000
Epoch 8/50
450/450 ————— 1s 2ms/step - loss: 11902923776.0000
- val_loss: 12453908480.0000
Epoch 9/50
450/450 ————— 1s 2ms/step - loss: 10919728128.0000
- val_loss: 11562933248.0000
Epoch 10/50
450/450 ————— 1s 3ms/step - loss: 10164074496.0000
- val_loss: 10793807872.0000
Epoch 11/50
450/450 ————— 1s 3ms/step - loss: 9516606464.0000 -
val_loss: 10112500736.0000
Epoch 12/50
450/450 ————— 2s 3ms/step - loss: 8937675776.0000 -
val_loss: 9484153856.0000
val_loss: 6247497728.0000
Epoch 20/50
450/450 ————— 1s 2ms/step - loss: 5892205568.0000 -
val_loss: 5972914176.0000
Epoch 21/50
450/450 ————— 1s 2ms/step - loss: 5693828608.0000 -
val_loss: 5748309504.0000
Epoch 22/50
450/450 ————— 1s 3ms/step - loss: 5531030016.0000 -
val_loss: 5558443008.0000
Epoch 23/50
450/450 ————— 2s 2ms/step - loss: 5390389760.0000 -
val_loss: 5401943040.0000
Epoch 24/50
450/450 ————— 1s 2ms/step - loss: 5271344128.0000 -
val_loss: 5274765824.0000
Epoch 25/50
450/450 ————— 1s 2ms/step - loss: 5170223616.0000 -
val_loss: 5167313408.0000
Epoch 26/50

450/450 ————— 1s 2ms/step - loss: 5082118144.0000 -
val_loss: 5069177344.0000
Epoch 27/50
450/450 ————— 1s 2ms/step - loss: 5004492800.0000 -
val_loss: 4988574208.0000
Epoch 28/50
450/450 ————— 1s 2ms/step - loss: 4938287104.0000 -
val_loss: 4913425408.0000
Epoch 29/50
450/450 ————— 1s 2ms/step - loss: 4881616384.0000 -
val_loss: 4856390144.0000
Epoch 30/50
450/450 ————— 1s 2ms/step - loss: 4832837120.0000 -
val_loss: 4805186560.0000
Epoch 31/50
450/450 ————— 1s 2ms/step - loss: 4790957056.0000 -
val_loss: 4764974592.0000
Epoch 32/50
450/450 ————— 1s 3ms/step - loss: 4754435072.0000 -
val_loss: 4728007680.0000
Epoch 33/50
450/450 ————— 2s 4ms/step - loss: 4724402176.0000 -
val_loss: 4691539456.0000
Epoch 34/50
450/450 ————— 1s 3ms/step - loss: 4696340992.0000 -
val_loss: 4664290816.0000
Epoch 35/50
450/450 ————— 1s 2ms/step - loss: 4671273472.0000 -
val_loss: 4638684672.0000
Epoch 36/50
450/450 ————— 1s 3ms/step - loss: 4649910272.0000 -
val_loss: 4616952320.0000
Epoch 37/50
450/450 ————— 1s 3ms/step - loss: 4630799872.0000 -
val_loss: 4596334080.0000
Epoch 38/50
450/450 ————— 1s 3ms/step - loss: 4613660160.0000 -
val_loss: 4580318720.0000
Epoch 39/50
450/450 ————— 1s 3ms/step - loss: 4598281728.0000 -
val_loss: 4563246080.0000
Epoch 40/50
450/450 ————— 1s 3ms/step - loss: 4584728064.0000 -
val_loss: 4549007872.0000
Epoch 41/50
450/450 ————— 1s 3ms/step - loss: 4573649920.0000 -
val_loss: 4537274880.0000
Epoch 42/50
450/450 ————— 2s 4ms/step - loss: 4560747008.0000 -
val_loss: 4525601280.0000

Epoch 43/50
 450/450 ————— 2s 3ms/step - loss: 4550177792.0000 -
 val_loss: 4518219264.0000
 Epoch 44/50
 450/450 ————— 1s 2ms/step - loss: 4540187648.0000 -
 val_loss: 4503932928.0000
 Epoch 45/50
 450/450 ————— 1s 3ms/step - loss: 4530190336.0000 -
 val_loss: 4493986304.0000
 Epoch 46/50
 450/450 ————— 1s 2ms/step - loss: 4521627136.0000 -
 val_loss: 4483959808.0000
 Epoch 47/50
 450/450 ————— 1s 2ms/step - loss: 4512576512.0000 -
 val_loss: 4481552384.0000
 Epoch 48/50
 450/450 ————— 1s 2ms/step - loss: 4505108480.0000 -
 val_loss: 4467888640.0000
 Epoch 49/50
 450/450 ————— 1s 2ms/step - loss: 4496734720.0000 -
 val_loss: 4464077312.0000
 Epoch 50/50
 450/450 ————— 1s 2ms/step - loss: 4489229824.0000 -
 val_loss: 4452781568.0000

Evaluating the model on the test set...

125/125 ————— 0s 2ms/step

Mean Squared Error (MSE) on the test set: 4514891799.5349

